Big Data y Minería de Datos

Unidad 2. Reglas en la minería de datos

U2.3. Reglas de asociación

UNIVERSIDAD DE
MURCIA

Unidad 2. Reglas en la minería de datos
Estructura

UNIVERSIDAD DE MURCIA

- Fundamentos de las reglas de asociación
- Descubrimiento de reglas de asociación
- Reglas multinivel
- Reglas multidimensionales
- Reglas de asociación secuenciales

Big Data y Minería de Datos
Contenidos

UNIVERSIDAD DE MURCIA

Introducción a la minería de datos

Técnicas de inteligencia artificial

Reglas
Fundamentos lógicos
Sistemas basados en reglas
Reglas de asociación

Aprendizaje computacional
Fundamentos y evaluación
Regresión y clasificación
No supervisado

Procesado del lenguaje natural
Fundamentos
Representación y tópicos
Sentimientos y NER

Introducción al big data

UNIVERSIDAD DE MURCIA



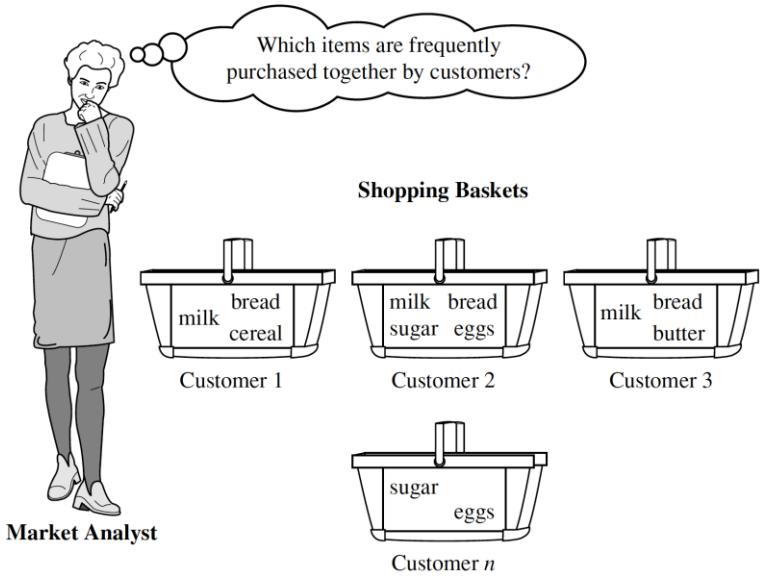
Unidad 2. Reglas en la minería de datos

FUNDAMENTOS DE LAS REGLAS DE ASOCIACIÓN

Unidad 2. Reglas en la minería de datos
¿Qué son las reglas de asociación?

UNIVERSIDAD DE MURCIA

- Expresan patrones de comportamiento entre datos, en función de aparición conjunta de dos o más atributos
- Son atributos nominales (no correlacionales con numéricos)
- Expresan combinaciones de valores que son más frecuentes
- Tienen importantes aplicaciones prácticas: análisis de la cesta de la compra, estudio de textos, búsqueda de patrones web…



- Inicialmente para afrontar el análisis de cestas de la compra
- Tratan los datos transaccionales, que se pueden representarse en una sola tabla
- Tenemos $I = \{i_1, i_2, \ldots, i_k\}$ que son los ítems que pueden aparecer en una transacción
  - una transacción, es básicamente el conjunto de ítems en un instante determinado
  - Si tiene *k* ítems, se le llama *k* itemset
- Por otra parte, tenemos el conjunto de transacciones $T = \{t_1, t_2, \ldots, t_n\}$
  - Que estará definida por el itemset que indica los ítems presentas en la transacción

- El formato de presentación más típico es mediante una disposición tabular binaria de una base de datos
- La celda $c_{i,j}$ corresponderá con un 1 o un 0 si en la transacción t_i se adquirió el ítem i_j o no respectivamente

	Vino	Gaseosa	Jamón	Detergente	Bizcochos	Ginebra
t_1	1	1	0	0	1	1
t_2	0	1	0	1	0	0
t_3	0	1	1	0	0	0
t_4	1	1	0	1	0	0
t_5	1	0	1	0	0	0
t_6	0	1	1	0	0	0
t_7	1	0	1	0	0	0
t_8	1	1	1	0	1	0
t_9	1	1	1	0	0	0

- También se puede tener una disposición horizontal de los datos
  - Podríamos representar t_1 como *{Vino, Gaseosa, Bizcochos}*, donde t_1 será un subconjunto de I ($t_1 \subseteq I$)
- También se puede realizar una representación vertical de los datos que sería similar pero transponiendo la tabla de transacciones

Unidad 2. Reglas en la minería de datos
¿Qué es una regla de asociación?

UNIVERSIDAD DE MURCIA

- Una regla puede ser vista de la forma SI α ENTONCES β, donde α y β deben de ser dos conjuntos de ítems disjuntos
  - Tenemos por lo tanto que α, β ⊆ I siendo $\alpha \cap \beta = \emptyset$ (conjunto vacio)
  - Por ejemplo: **SI** vino **Y** gaseosa **ENTONCES** jamón
  - Esto también puede ser expresado como una implicación $\alpha \Rightarrow \beta$ que conecta con nuestro tema anterior
  - Proposición lógica con **factor probabilístico**
  - Indica que existe una fuerte relación entre la venta de vino y gaseosa con la venta de jamón
- Importante denotar que una regla de asociación no implica causalidad, sino co-ocurrencia

Soporte

- Dada una regla $\alpha \Rightarrow \beta$, su soporte es la fracción de transacciones en las que están incluidos los itemsets del antecedente y consecuente ($\alpha \cup \beta$)
  - $Sop(\alpha \Rightarrow \beta) = Sop(\alpha \cup \beta) = \frac{|\alpha \cup \beta|}{|T|}$
  - Se puede ver también como la fracción de transacciones donde la regla predice correctamente
  - Otra interpretación es la probabilidad de que α y β ocurran a la vez $Pr(\alpha \cup \beta)$
- El soporte de una regla nos indica cuantas veces es aplicable dentro del conjunto de transacciones
  - ¿Como influye en su utilidad?
- Sop(vino & gaseosa ⇒ jamón) = 2/9

Unidad 2. Reglas en la minería de datos
Confianza

- Dada una regla $\alpha \Rightarrow \beta$, su confianza es la fracción de transacciones en las que aparece el itemset β (consecuente) y que también incluye al itemset α (antecedente)
  - $\text{Conf}(\alpha \Rightarrow \beta) = \frac{\text{Sop}(\alpha \cup \beta)}{\text{Sop}(\alpha)} = \frac{|\alpha \cup \beta|}{|\alpha|}$
  - Puede interpretarse como el porcentaje de veces en el que la regla se cumple
  - Otra interpretación es la probabilidad de que β ocurra cuando lo hace α $\Pr(\beta|\alpha)$
- Nos da una medida de lo frecuente que β aparezca cuando la transacción contiene α
  - ¿Cómo influye en su utilidad?
- Conf(vino & gaseosa ⇒ jamón) = 2/4

- Reglas de asociación son un caso específico minería de conjunto de ítems frecuentes
- En base a la clasificación de [Han & Kamber 2001]:
  - **Tipos de valores utilizados en reglas**: Atributos binarios (como cesta de la compra), pero también con atributos con más valores, **SI** país=Alemania **ENTONCES**… o **SI** edad > 18 **ENTONCES**…
  - **Dimensiones de los datos**: Regla multidimensional (no sólo compra) - **SI** Comprar (vino, Cliente(Juan), Tiempo(Marzo) **ENTONCES** Comprar(gaseosa)
  - **Niveles de abstracción**: Niveles de abstracción con concepto que aglutinan otros, reglas multi-nivel - **SI** Comprar(bebida) **ENTONCES** Comprar(bizcochos)
  - **Instantáneas o secuenciales**: Relaciones en un instante de tiempo (determinada compra) o secuencia/serie (varias compras, visitas web)

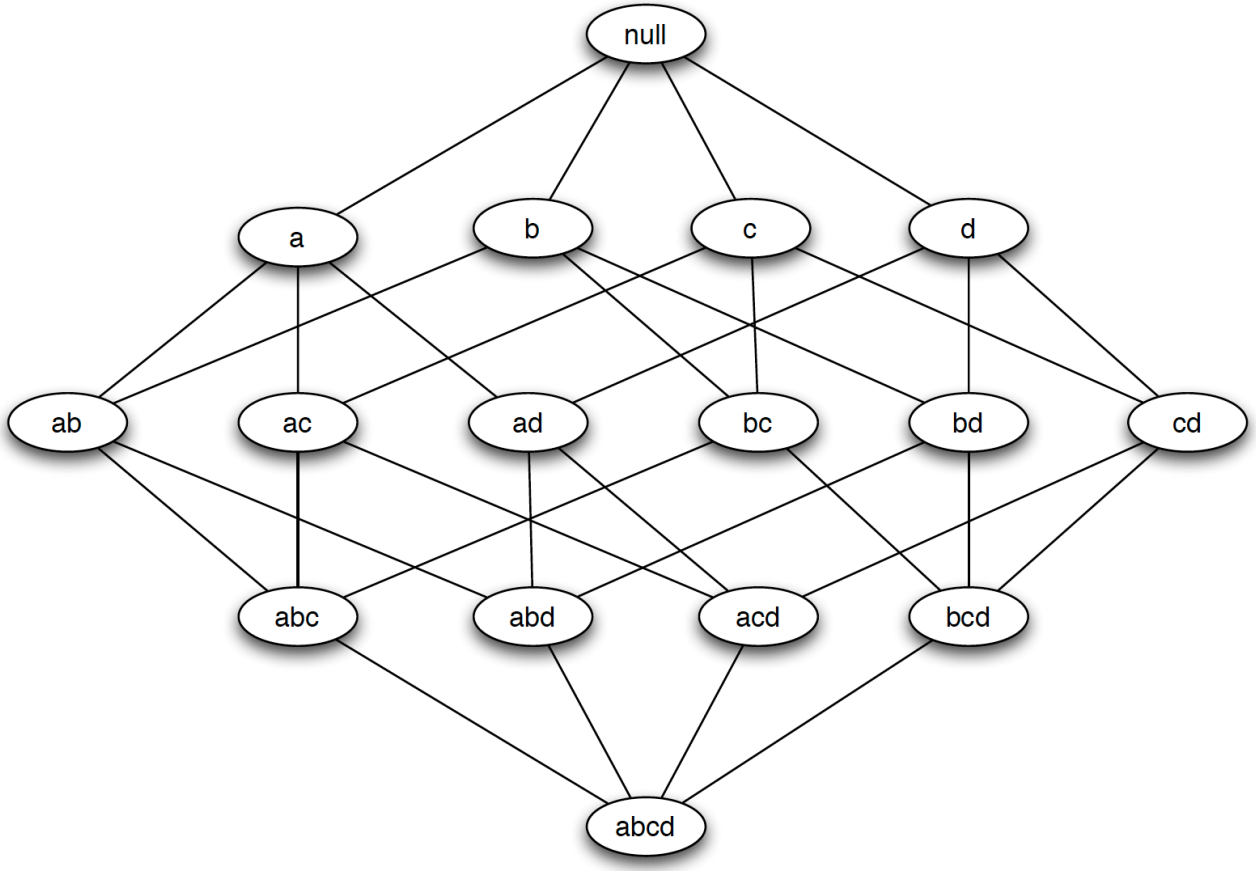
UNIVERSIDAD DE MURCIA



Unidad 2. Reglas en la minería de datos

DESCUBRIMIENTO DE REGLAS DE ASOCIACIÓN

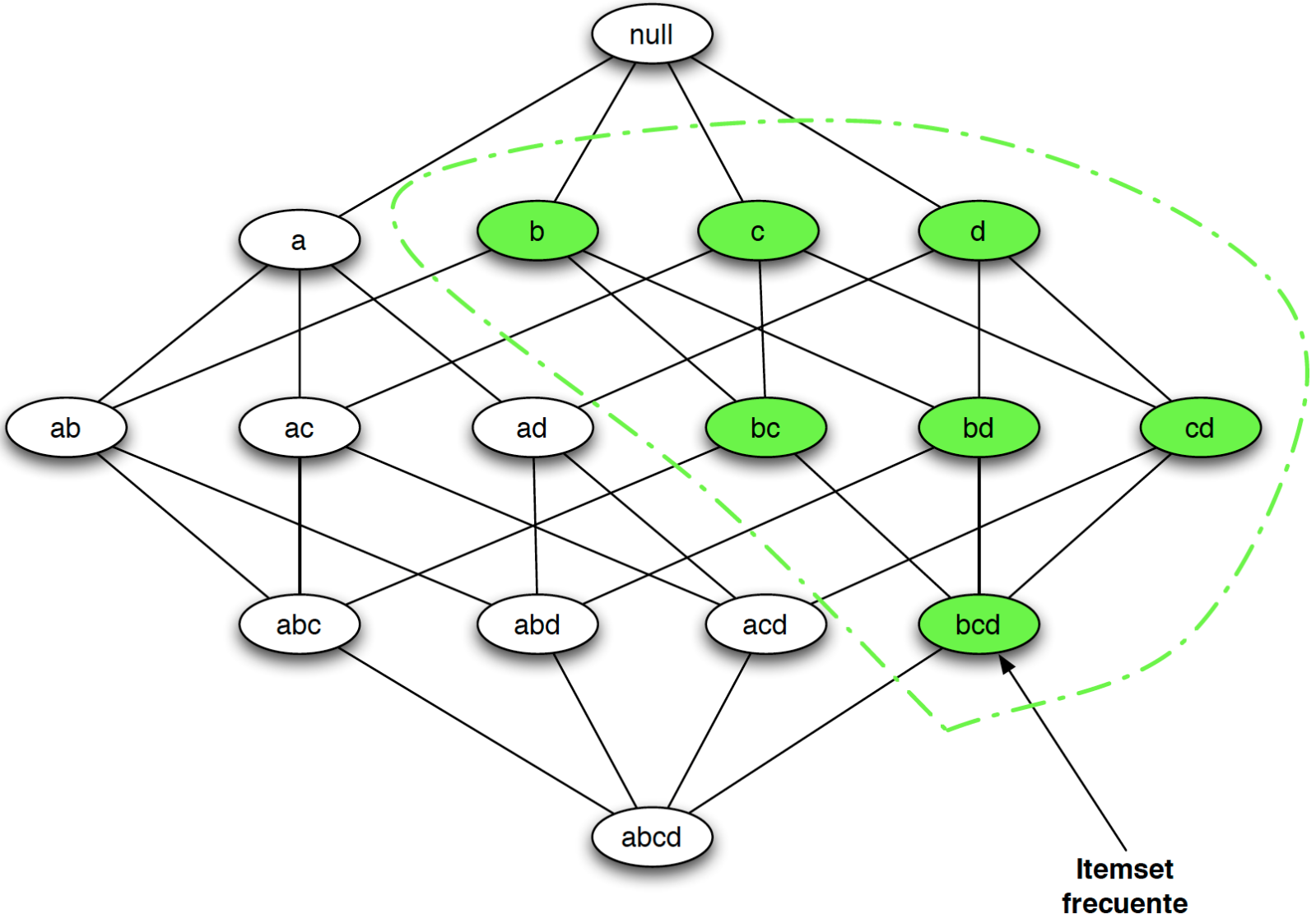
- Si tenemos un conjunto de datos con *k* ítems ($I = \{i_1, i_2, \ldots, i_k\}$), se podrían generar $2^k - 1$ *itemsets* frecuentes
- El espacio de búsqueda crece exponencialmente con *k*
  - Por ejemplo, para un $I = \{a, b, c, d\}$



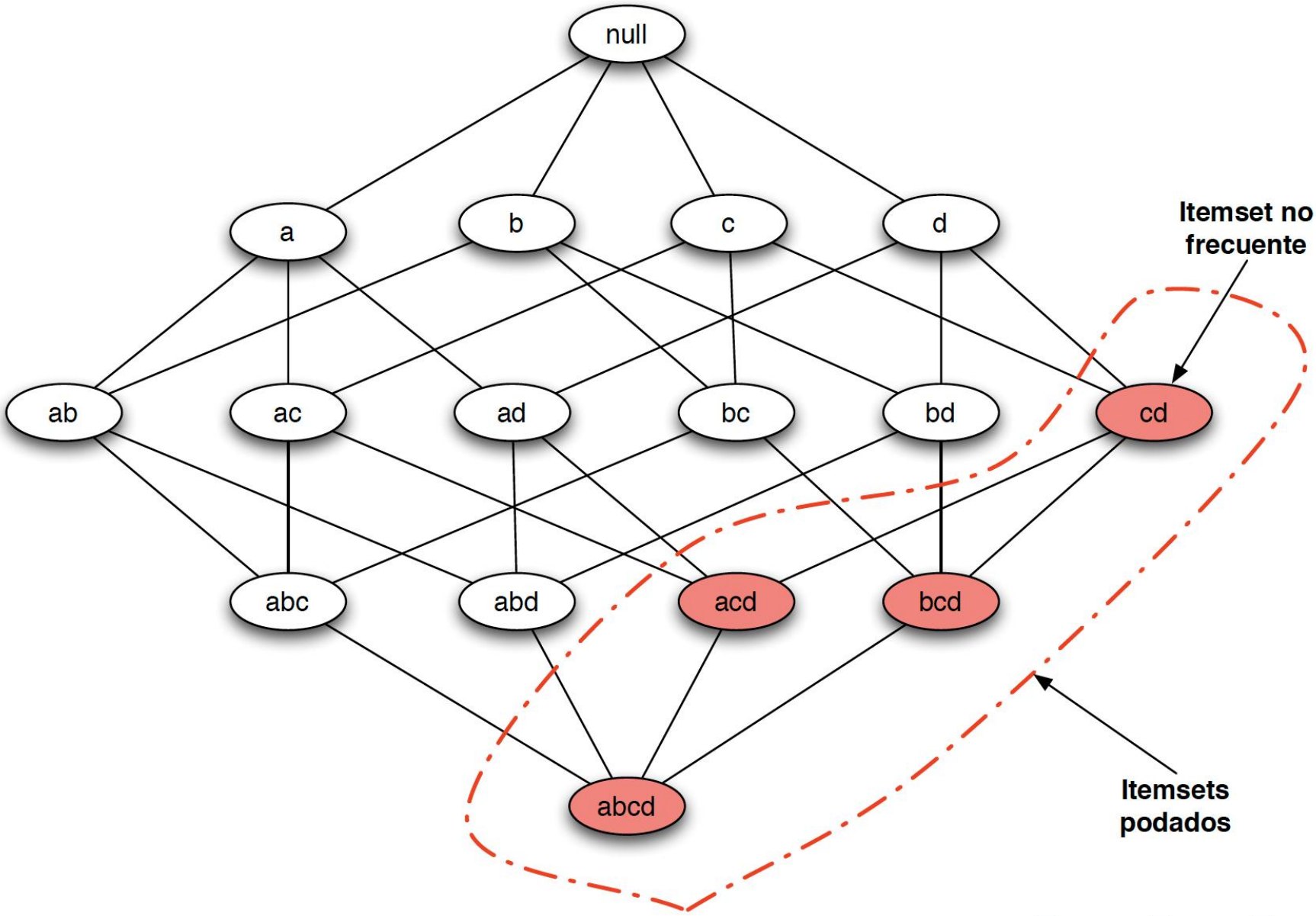
- Hay dos estrategias para reducir esta carga computacional
  - Reducir el número de *itemsets* candidatos: Se reducen los candidatos sin tener que calcular su soporte
    - Se hace mediante el principio Apriori que veremos ahora
  - Reducir el número de comparaciones: Se basa en la utilización de estructuras de datos avanzadas que reducen el número de comparaciones necesarias

- Principio: Todos los subconjuntos no vacíos de un *itemset* frecuente son también frecuentes
- Si un *itemset I* es frecuente (alcanza *minsop*), cualquier adición *A* será también frecuente (al menos en todas las transacciones que aparece *I*)
- Sop(I) >= minsop → Sop($I \cup A$) >= minsop
- Si un *itemset I* no alcanza el *minsop*, si se añade un ítem A, no puede ser mayor que *I*. Por lo tanto, $Sop(I \cup A) <$ *minsop*
  - Esta propiedad pertenece a una categoría especial llamada anti-monotonicidad
- Este principio se va a usar para unir o podar

Principio Apriori: Itemsets frecuentes



Unidad 2. Reglas en la minería de datos
Principio Apriori: Poda mediante no frecuentes



- El proceso del algoritmo se realiza en dos fases:

1. **Generación de candidatos**: Se generan los *k*-itemsets candidatos, basándonos en los **(*k*-1)-*itemsets* frecuentes** encontrados en la anterior interacción
2. **Poda de candidatos**: Se eliminan los *k-itemsets* candidatos que no alcancen el *minsop*



Apriori(T: datos, minsop: soporte mínimo):

k = 1

Rellena_Item(C_i) //Incluye en C_1 todos los ítems de tamaño 1

MIENTRAS $C_k \neq \emptyset$

PARA CADA x=elemento de C_k

SI soporte(x) ≥ minsop **ENTONCES** $L_k = L_k \cup x$

FINPARA

C_{k+1}=Selecciona_Candidatos (L_k)

k = k + 1;

FIN MIENTRAS

RETORNA C

FIN ALGORITMO

UNIVERSIDAD DE MURCIA

1. En la primera iteración del algoritmo, cada ítem es un miembro del 1-itemset candidato, C_1. Escanea la cuenta de cada miembro directamente calculando soporte

2. Calculamos L_1 con el *minsop* (por ej 2, en términos absolutos), en nuestro caso relativos 2/9 (transacciones)

1 **C_1**

1-itemset	Soporte
{Vino}	6
{Gaseosa}	7
{Jamón}	6
{Detergente}	2
{Bizcochos}	2
{Ginebra}	1

2 **L_1**

1-itemset	Soporte
{Vino}	6
{Gaseosa}	7
{Jamón}	6
{Detergente}	2
{Bizcochos}	2

UNIVERSIDAD DE MURCIA

3. Para generar el 2-itemset, L_2, el algoritmo calcula L_1 x L_1 con los 2-itemsets. No se podan candidatos porque cada subconjunto también era frecuente en L_1

(selección de candidatos)

3

C_2

2-itemset	Soporte
{Vino, gaseosa}	?
{Vino, jamón}	?
{Vino, detergente}	?
{Vino, bizcochos}	?
{Gaseosa, jamón}	?
{Gaseosa, detergente}	?
{Gaseosa, bizcochos}	?
{Jamón, detergente}	?
{Jamón, bizcochos}	?
{Detergente, bizcochos}	?

UNIVERSIDAD DE MURCIA

4. Se calcula el soporte para todos los 2-itemsets. No se podan candidatos porque cada subconjunto también era frecuente en L_1

4

C_2

2-itemset	Soporte
{Vino, gaseosa}	4
{Vino, jamón}	4
{Vino, detergente}	1
{Vino, bizcochos}	2
{Gaseosa, jamón}	4
{Gaseosa, detergente}	2
{Gaseosa, bizcochos}	2
{Jamón, detergente}	0
{Jamón, bizcochos}	1
{Detergente, bizcochos}	0

UNIVERSIDAD DE MURCIA

5. Calculamos L_2 verificando el soporte de los 2-itemsets para ver cuales alcanzan *minsop* = 2

5

L_2

2-itemset	Soporte
{Vino, gaseosa}	4
{Vino, jamón}	4
{Vino, bizcochos}	2
{Gaseosa, jamón}	4
{Gaseosa, detergente}	2
{Gaseosa, bizcochos}	2

UNIVERSIDAD DE MURCIA

6. Para generar el 3-itemset, C_3, el algoritmo calcula L_2 x L_2 con los 2-itemsets

6a) Unión, los unimos todos sin calcular soporte (prefijo k-2 común)

6b) Aplicamos poda a aquellos que estén formados por 2-itemsets no frecuentes en L_2

6a

3-itemset	Soporte
{Vino, gaseosa, jamón}	?
{Vino, gaseosa, bizcochos}	?
{Vino, jamón, bizcochos}	?
{Gaseosa, jamón, detergente}	?
{Gaseosa, jamón, bizcochos}	?
{Gaseosa, detergente, bizcochos}	?

6b

3-itemset	Soporte
{Vino, gaseosa, jamón}	?
{Vino, gaseosa, bizcochos}	?
{Vino, jamón, bizcochos}	?
{Gaseosa, jamón, detergente}	?
{Gaseosa, jamón, bizcochos}	?
{Gaseosa, detergente, bizcochos}	?

UNIVERSIDAD DE MURCIA

6c) Por lo tanto, después de la poda C_3 queda como sigue:

6c

C_3

3-itemset	Soporte
{Vino, gaseosa, jamón}	?
{Vino, gaseosa, bizcochos}	?

7) Calculamos L_3 verificando el soporte de los 3-itemsets para ver cuales alcanzan *minsop* = 2

7

L_3

3-itemset	Soporte
{Vino, gaseosa, jamón}	2
{Vino, gaseosa, bizcochos}	2

UNIVERSIDAD DE MURCIA

8a) Para generar el 4-itemset, C_4, el algoritmo calcula L_3 x L_3 con los 3-itemsets

8a

4-itemset	Soporte
{Vino, gaseosa, jamón, bizcochos}	?

8b) Pero este se poda porque tiene un 3-itemset no frecuente {vino, jamón, bizcochos} en L_3

8b

4-itemset	Soporte
{Vino, gaseosa, jamón, bizcochos}	?

8c) $C_4 = \emptyset$

El algoritmo termina.

Itemsets frecuentes = $L_1 \cup L_2 \cup L_3$

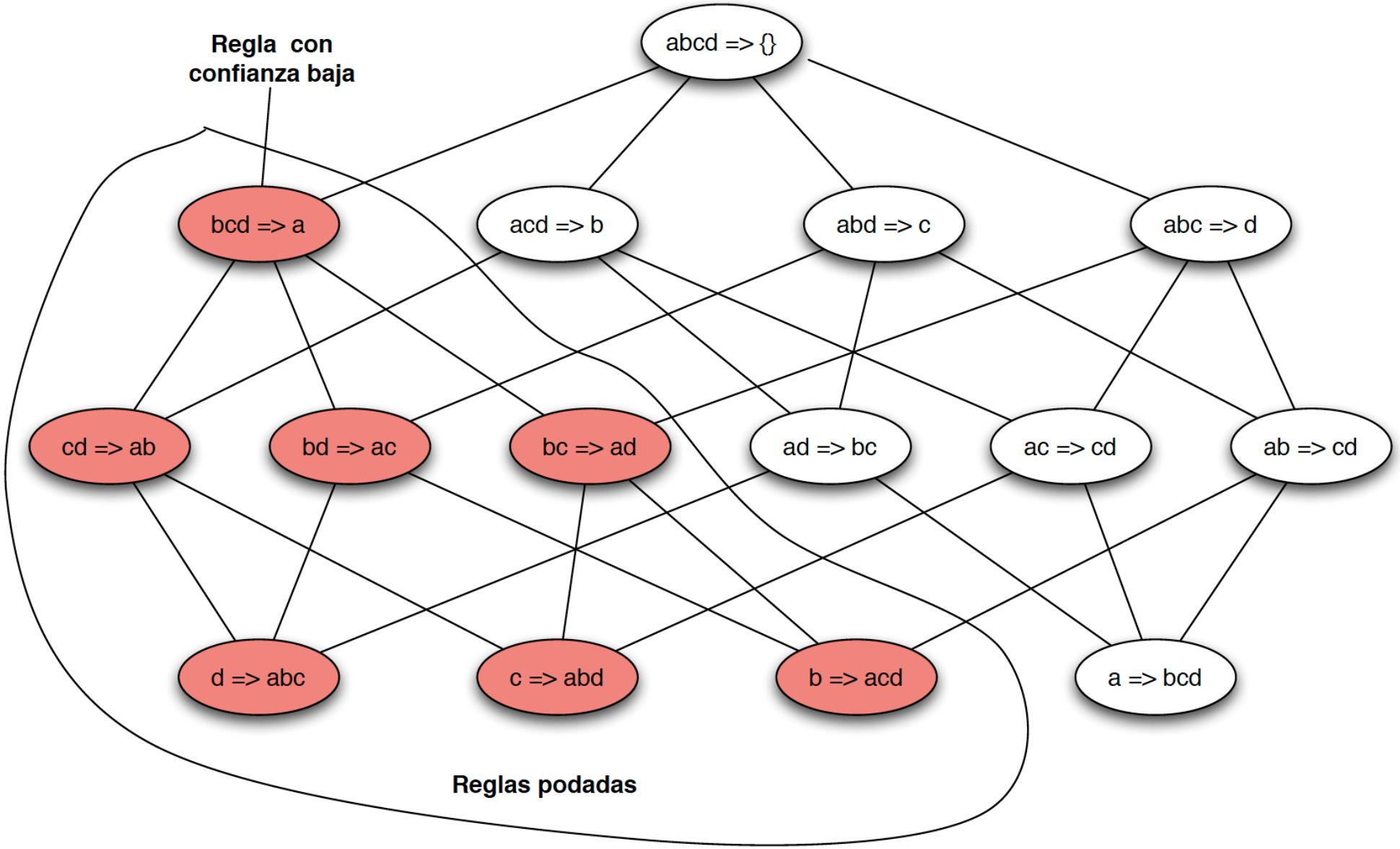
- Una vez generados los itemsets frecuentes, calculamos las reglas de asociación fuertes
  - Entendemos fuertes como aquellas que satisfacen un soporte y confianza mínimos
  - $\text{Conf}(\alpha \Rightarrow \beta) = \Pr(\beta|\alpha) = \frac{\text{Sop}(\alpha \cup \beta)}{\text{Sop}(\alpha)} = \frac{|\alpha \cup \beta|}{|\alpha|}$
- Por cada *k-itemset* frecuente, A, se pueden generar 2^k-2 reglas de asociación
  - Las reglas con conjuntos nulos no se tienen en cuenta
- Para crear una regla de asociación a partir de I:
  1. Dividir I en dos conjuntos disjuntos B y A \ B
  2. Generar la regla B ⇒ A \ B (si supera la *minconf*)

- Ejemplo: Tenemos el 3-itemset frecuente {Vino, gaseosa, jamón}, resumido como {v, g, j}. Se pueden sacar las siguientes reglas (2^3 -2 = 6 reglas):
  - {v, g} ⇒ {j}, {v, j} ⇒ {g}, {g, j} ⇒ {v}
  - {j} ⇒ {v, g}, {g} ⇒ {v, j}, {v} ⇒ {g, j}
- El soporte de cada regla supera *minsop*, ya que fue generada por un *itemset* frecuente
- Calcular la confianza no hace necesario volver a recorrer la base de datos (están en los L_k)

$$\text{Conf}(\alpha \Rightarrow \beta) = \frac{\text{Sop}(\alpha \cup \beta)}{\text{Sop}(\alpha)}$$

- La medida de la confianza no cumple ninguna propiedad de la monotonicidad
- Sin embargo, dada una regla $B \Rightarrow A \setminus B$ cuya confianza es menor que *minconf*, otras reglas como $B' \Rightarrow A \setminus B'$, en donde $B' \subseteq B$, tendrán confianza por debajo de *minconf*
- Se puede usar esta propiedad para podar conjuntos de reglas

Unidad 2. Reglas en la minería de datos
Poda basada en la confianza



- Sigue un proceso por niveles en base al número de ítems en el consecuente
- El algoritmo comienza por aquellas reglas que tienen un solo ítem en el consecuente y nos quedamos con las que tienen una confianza alta
  - Usamos las de baja confianza para podar el siguiente nivel (con consecuentes mayores)
- Esas reglas se utilizarán para generar las del siguiente nivel

Generación reglas de asociación en Apriori: Ejemplo

- Ejercicio anterior terminó con los siguientes itemsets frecuentes:
  - 3-itemsets: {Vino, gaseosa, jamón} y {Vino, gaseosa, bizcochos} (todos los subconjuntos también frecuentes)

1. Generamos las reglas de primer nivel (1 item en el consecuente)

1

Reglas primer nivel	Confianza
{Vino, gaseosa} ⇒ {jamón}	?
{Vino, jamón} ⇒ {gaseosa}	?
{Jamón, gaseosa} ⇒ {vino}	?
{Vino, gaseosa} ⇒ {bizcochos}	?
{Vino, bizcochos} ⇒ {gaseosa}	?
{Bizcochos, gaseosa} ⇒ {vino}	?

UNIVERSIDAD DE MURCIA

2. Se calcula la confianza de todas las reglas

2

Reglas primer nivel	Confianza
{Vino, gaseosa} ⇒ {jamón}	2/4 = 50%
{Vino, jamón} ⇒ {gaseosa}	2/4 = 50%
{Jamón, gaseosa} ⇒ {vino}	2/4 = 50%
{Vino, gaseosa} ⇒ {bizcochos}	2/4 = 50%
{Vino, bizcochos} ⇒ {gaseosa}	2/2 = 100%
{Bizcochos, gaseosa} ⇒ {vino}	2/2 = 100%

UNIVERSIDAD DE MURCIA

3. Nos quedamos sólo con aquellas reglas que alcancen *minconf*, en este caso, supongamos 75%

3

Reglas primer nivel	Confianza
{Vino, gaseosa} ⇒ {jamón}	2/4 = 50%
{Vino, jamón} ⇒ {gaseosa}	2/4 = 50%
{Jamón, gaseosa} ⇒ {vino}	2/4 = 50%
{Vino, gaseosa} ⇒ {bizcochos}	2/4 = 50%
{Vino, bizcochos} ⇒ {gaseosa}	2/2 = 100%
{Bizcochos, gaseosa} ⇒ {vino}	2/2 = 100%

Generación reglas de asociación en Apriori: Ejemplo

4. Ahora se genera el siguiente nivel de reglas en base a las dos reglas obtenidas

4

Reglas primer nivel	Confianza
{Gaseosa} ⇒ {vino, bizcochos}	?
{Vino} ⇒ {bizcochos, gaseosa}	?
{Bizcochos} ⇒ {vino, gaseosa}	?

5. Se realiza poda en base a la regla {Vino, gaseosa} ⇒ {bizcochos} que *minconf* no se alcanzaba (monotonicidad)

5

Reglas primer nivel	Confianza
{Gaseosa} ⇒ {vino, bizcochos}	?
{Vino} ⇒ {bizcochos, gaseosa}	?
{Bizcochos} ⇒ {vino, gaseosa}	?

UNIVERSIDAD DE MURCIA

6. Se calcula la confianza de las reglas no podadas

6

Reglas primer nivel	Confianza
{Bizcochos} ⇒ {vino, gaseosa}	2/2 = 100%

7. Se comprueban se alcanza el umbral de *minconf* del 75%

7

Reglas primer nivel	Confianza
{Bizcochos} ⇒ {vino, gaseosa}	2/2 = 100%

8. Se obtienen las reglas finales a través del algoritmo:

{Vino, bizcochos} ⇒ {gaseosa} (soporte = 2/9, confianza = 100%)

{Bizcochos, gaseosa} ⇒ {vino} (soporte = 2/9, confianza = 100%)

{Bizcochos} ⇒ {vino, gaseosa} (soporte = 2/9, confianza = 100%)

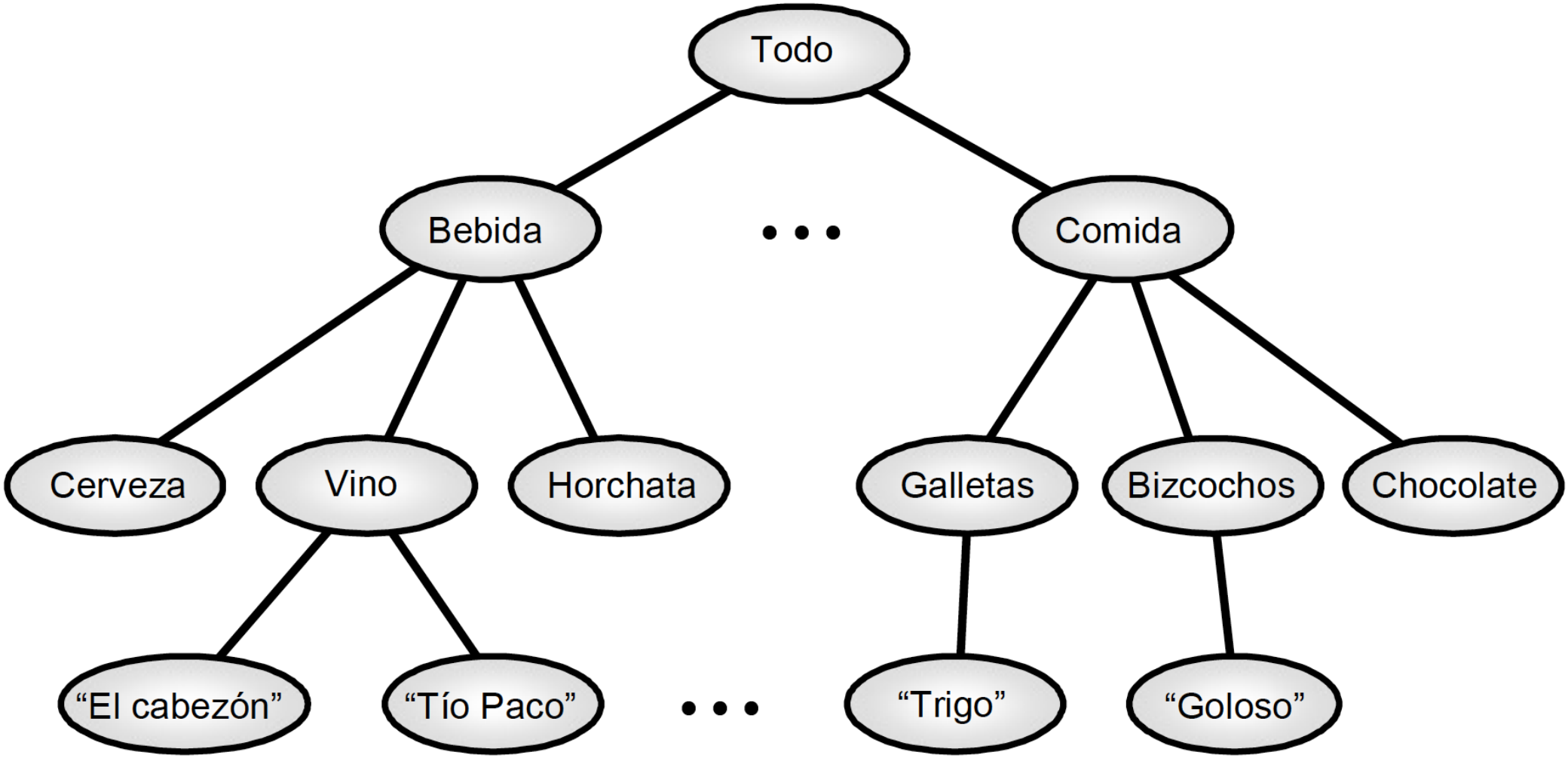
UNIVERSIDAD DE MURCIA

Unidad 2. Reglas en la minería de datos

REGLAS MULTINIVEL

- Con datos muy dispersos es difícil encontrar (reglas de) asociaciones
  - Hay muchos ítems con pocas presencias en cada transacción
- Una mejora es la agrupación de ítems en categorías
  - El aprendizaje se hace con categorías (mejor confianza y soporte)
  - Se las llama reglas de asociación multinivel
- Se debe proporcionar una jerarquía de conceptos que los ítems se agrupen en distintos niveles de abstracción
- En el nivel superior están todos los conceptos y en el nivel inferior los ítems
- Nivel de agrupamiento flexible, para evitar reglas demasiado genéricas (si compra comida, compra bebida) o específicas (con baja cobertura)

Unidad 2. Reglas en la minería de datos
Ejemplo de jerarquía de conceptos



Reglas multinivel: Algoritmo

- Se ejecuta de forma bastante similar al clásico
- Se buscan reglas fuertes (cumplen con *minsop* y *minconf)*, haciendo la búsqueda desde conceptos más abstractos a más específicos
- Se empieza por el nivel 1 hasta el nivel más bajo del arblo de jerarquía (donde están los ítems)
- Para cada nivel, se puede aplicar cualquier algoritmo de búsqueda como por ejemplo, el algoritmo Apriori

- **Soporte uniforme**: se utiliza el mismo límite (de soporte minsop) para todos niveles
  - Aproximación simple
  - Optimiza búsqueda debido a que si concepto de nivel superior que no cumple el minsop, entonces no tendrá inferiores que lo cumplan (minsop)
  - La misma regla de la antimonotonicidad que se vio antes
  - Pero es poco realista, porque los de niveles bajos van a ocurrir mucho menos frecuentemente
  - Poniendo el umbral muy alto o muy bajo podemos estar perdiendo relaciones de interés en niveles bajos de abstracción o generando relaciones no interesantes

- **Soporte reducido**: evolución del anterior (soporte uniforme), se establece un *minsop* específico en cada nivel de la jerarquiía
  - Normalmente se va incrementando cuando desciende la jerarquía con conceptos menos abstractos
  - El problema es que la búsqueda no es tan sencilla, ya que al ampliar el límite nivel a nivel, no se puede aplicar anti-monotonicidad (del soporte uniforme)
- **Soporte basado en grupos**: Cuando se dispone de información importante/destacada, se puede ajustar el umbral (minsop) por ítems o grupos

UNIVERSIDAD DE MURCIA

Unidad 2. Reglas en la minería de datos

REGLAS MULTIDIMENSIONALES



Reglas unidimensioanes o intradimensionales

- Hasta este momento, nos hemos centrado en reglas con la presencia o no en cada transacción
- Esto ha permitido generar reglas con un único predicado, por ejemplo:
  - Compra(X, vino) ⇒ Compra(X, gaseosa)
- Siguiendo la terminología de los almacenes de datos multidimensionales, llamaremos a cada predicado como dimensión
- Esto es lo que conocemos como reglas unidimensionales o intradimensionales

Reglas multidimensionales o interdimensionales

- Los almacenes de datos multidimensionales almacenan mucho más información, que simplemente las transacciones
  - Puede haber muchas otras dimensiones, como la descripción del ítem o la localización en/de la tienda
  - Podría haber información de los clientes que compraron el ítem
- Si consideramos cada dimensión como un predicado, podríamos hacer reglas como:
  - Edad(X, “65…75”) ^ ocupación(X, “jubilado”) ⇒ Compra(X, ”vino”)
- Llamaremos a estas reglas que **no repiten predicados** como multidimensionales o interdimensionales

Reglas de dimensionalidad híbrida

- Las reglas de dimensionalidad híbrida, son aquellas que pueden repetir algún predicado
  - Edad(X, “65…75”) ^ Compra(X, “gaseosa”) ⇒ Compra(X, ”vino”)

- Los atributos pueden ser nominales o cuantitativos
  - Los nominales (o categóricos) son “nombres de cosas” y tienen un número finito de valores (ocupación, marca, color…)
  - Los cuantitativos (o numéricos) tienen un orden implícito (edad, ganancias, precio…)

- Las técnicas para minar este tipo de reglas multidimensionales pueden ser de dos tipos, en función del tratamiento de los atributos cuantitativos (o numéricos)
  - Discretización estática: Se realiza al principio del proceso y no se modifica
    - Por ejemplo, para establecer rangos en el salario 0..20k, 21k..30k, 31…40k…
  - Discretización dinámica: La discretización puede cambiar conforme va avanzando el proceso
    - Por ejemplo, para satisfacer un mínimo criterio de confianza

UNIVERSIDAD DE MURCIA

Unidad 2. Reglas en la minería de datos

REGLAS DE ASOCIACIÓN SECUENCIALES

- Expresan patrones de comportamiento secuenciales
  - Distintos instantes de tiempo, pero cercanos
- Es crucial en diversas áreas, como navegación web o en aplicaciones
  - Por ej, 40% que visitan la cartelera, compran en dos días las entradas
- Permite mejorar el diseño de las webs o apps
- También puede ser útil en el comercio electrónico para ventas temporales
  - Por ej, si cliente compra frigorífico y microondas, en menos de seis meses compra lavadora
- Ahora si va a tener una importancia clara, las transacciones del mismo usuario

UNIVERSIDAD DE MURCIA

- Se necesitan tablas de transacciones con clientes, instantes de tiempo e ítems
- De estas tablas se pueden extraer secuencias
- En este caso, la cobertura será la proporción de clientes que contiene determinada secuencia
- Objetivo: Encontrar secuencias que cumplan *minsop*

ID-Cliente	ID-Tiempo	Items
1	15/03/2023	{23,56}
1	17/03/2023	{42,13}
1	18/03/2023	{45,33}
2	12/03/2023	{12,13}
2	18/03/2023	{23,34,5,8}

ID-Cliente	Secuencia
1	<{23 56}{42 13}{45 33}>
2	<{{12 13}{23 34 5 8}>

Unidad 2. Reglas en la minería de datos
Algoritmo AprioriAll

- El algoritmo AprioriAll es de los mas populares para encontrar reglas secuenciales
- Tiene cinco fases que vamos a ver con la base de datos de ejemplo de la derecha
  - Ordenación
  - Selección
  - Transfomación / Renombramiento
  - Secuencias Frecuentes
  - Secuencias Máximas

ID-Cliente	ID-Tiempo	Ítems
1	15/03/2023	{30}
1	17/03/2023	{90}
2	18/03/2023	{10,20}
2	12/03/2023	{30}
2	18/03/2023	{40,60,70}
3	18/03/2023	{30,50,70}
4	13/03/2023	{30}
4	15/03/2023	{40,70}
4	17/03/2023	{90}
5	14/03/2023	{90}

Algoritmo AprioriAll: Ordenación

1. Se ordena la base de datos con ID-Cliente como la clave primaria y el ID-Tiempo como la secundaria.

Se generan secuencia de conjuntos de ítems para cada cliente

1

ID-Cliente	Secuencia
1	<{30} {90}>
2	<{10,20} {30} {40,60,70}>
3	<{30,50,70}>
4	<{30} {40,70} {90}>
5	<{90}>

Algoritmo AprioriAll: Selección

2. Consiste en terminar los conjuntos de ítems con una mínima cobertura con respecto a las secuencias por cliente

Si tomamos una cobertura de mínima igual a 2 entonces:

2

Conjuntos de ítems	Identificador
{30}	1
{40}	2
{70}	3
{40 70}	4
{90}	5

UNIVERSIDAD DE MURCIA

3. Se le asigna a cada conjunto de ítems un identificador.

Se transforma para que sólo tenga ítems frecuentes

Se renombra por su identificador

3

ID-Cliente	Secuencia	Secuencia transformada	Sec. Trans. y renombrada
1	<{30} {90}>	<{{30}}{{90}}>	<{1}{5}>
2	<{10,20} {30} {40,60,70}>	<{{30}}{{40}, {70}, {40,70}}>	<{1}{2,3,4}>
3	<{30,50,70}>	<{{30}, {70}}>	<{1,3}>
4	<{30} {40,70} {90}>	<{{30}}{{40},{70},{40 70}}{{90}}>	<{1}{2,3,4}{5}>
5	<{90}>	<{{90}}>	<{5}>

4. A partir de ítems frecuentes, se construyen incrementalmente el conjunto de secuencias frecuentes que cumplan *minsup*

4

Tamaño 1		Tamaño 2		Tamaño 3		Tamaño 4	
Secuencia	**Soporte**	**Secuencia**	**Soporte**	**Secuencia**	**Soporte**	**Secuencia**	**Soporte**
<1>	4	<1 2>	2	<1 2 3>	2	<1 2 3 4>	2
<2>	2	<1 3>	3	<1 2 4>	2		
<3>	3	<1 4>	2	<1 3 4>	2		
<4>	2	<1 5>	2	<2 3 4>	2		
<5>	3	<2 3>	2				
		<2 4>	2				
		<3 4>	2				

UNIVERSIDAD DE MURCIA

5. Filtra el conjunto de secuencias frecuentes para evitar que haya subsecuencias de mayor tamaño

5

Secuencia	Secuencia original	Cobertura
<1 2 3 4>	<{30} {40} {70} {40 70}>	2
<1 5>	<{30} {90}>	2